pyprims quick reference

Session ⊃ Model ⊃ Tasks · trial = one run of a task's script

Imports and command line

```
from pyprims import Model, Task, load_task, run_trials
from pyprims.notebook import Session, Dashboard
```

`python -m pyprims run f.prims -n 10`	run n trials (--trace 3)
`python -m pyprims batch f.bprims -o out.dat`	batch script
`python -m pyprims convert f.prims`	→ f.py

Model (scripts)

`m = Model(seed=None, trace=True)`	new model; trace=False is faster
`m.load(task)`	.prims / .py path or Task; extra tasks = transfer
`m.switch_task(name)`	make a loaded task current
`m.run(n=1)`	n trials of the current task
`m.run_trial()`	script once
`m.set_parameter("rt", "-2.0")`	PRIMs names, string values
`m.results`	per trial: time or plot-point
`m.output_data`	batch events
`m.get_trace(level=3)`	last trial, levels 0–5
`m.dm.chunks`	dict of chunks
`m.chunk("cf1").activation()`	one chunk
`m.operators_in_dm()`	operator chunks
`m.production_table()`	productions + utilities

Defining a task in Python

```
task = Task("count", initial_skills=["count"], rt=-2.0)
sk = task.skill("count")
sk.operator("start", """V1 <> nil ==> V1 -> WM1 ...""")
task.facts([("cf1", "count-fact", "one", "two")])
task.action("say", latency=0.3, noise=0.1)
@task.script
def script(m):                    # m = script interface
    m.screen("one", "three")
    m.run_until_action("say", "stop")
    m.issue_reward(); m.trial_end()
```

Parameters: `-` → _, `t` → `True`, `nil` → `False`. `load_task(path)` builds a Task from a file; a .py file must define `task`.

Script functions (m....)

Running

`run_step()` · `run_until_action(*a)` · `run_relative_time_or_action(t, *a)` · `run_absolute_time_or_action(t, *a)` · `sleep(t)` · `time()` · `trial_start()` · `trial_end()` · `issue_reward(r=None)` · `last_action()`

Input and output

`screen(*items)` · `add_visual(name, *slots)` · `print(*a)` · `data_line(*a)` · `set_data_file_field(i, v)` · `plot_point(y)` · `set_graph_title(t)` · `set_average_window(n)` · `trace_operators(v)` · `read_file(f)`

Memory, goals, buffers

`add_dm(name, *slots)` · `set_activation(c, v)` · `get_activation(c)` · `set_references(c, n)` · `set_sji(…)` · `get_sji(c, buf, slot, src)` · `sgp(name, v)` · `imaginal_to_dm()` · `report_memory()` · `set_goal(name, …)` · `create_new_goal(name, …)` · `instantiate_skill(skill, inst, …)` · `add_binding(attr, v)` · `purge_bindings()` · `set_buffer_slot(b, s, v)` · `get_buffer_slot(b, s)`

Utilities

`random(n)` · `shuffle(lst)` · `length(x)` · `int(x)` · `str_to_int(s)` · `random_string(prefix)` · `arg` · `state` (dict for the script's own data)

Batch runs

`run_trials(m, n, task=None, **fields)`	list of dicts per trial end
`run_trials(…, events=True)`	one dict per event
`run_batch("x.bprims", out_path, seed)`	output lines

```
rows = []
for rep in range(10):
    m = Model(seed=rep, trace=False)
    rows += run_trials(m, 100, "count.prims", rep=rep)
    rows += run_trials(m, 100, "semantic.prims", rep=rep)
data = pd.DataFrame(rows)  # trial task result outcome rep
```

Session (notebooks)

```
s = Session(seed=1)        # contains the Model: s.model
s.load("count.prims"); s.run(100); s.plot_results()
Dashboard(s)               # Swift GUI as widgets
```

Loading and control

`s.load(source, name=None)`	path, Task or model text; same name = redefine + reset
`s.load_prims(text) / s.load_python(code)`	model text
`s.select_task(name)`	make task current
`s.reset() / s.clear()`	fresh model / forget all
`s.step(n=1)`	n half steps
`s.step_operator(n=1)`	until n operators done
`s.finish_trial()`	finish stepped trial
`s.run(n=1) / s.run_time(sec)`	trials / model seconds
`s.stop()`	stop a run
`s.batch("x.bprims", out_path, seed)`	batch, own model

Inspecting

`s.status()`	task, time, counts
`s.trace(level) / s.show_trace(level)`	0 output … 5 all
`s.buffers() / s.conflict_set()`	buffers / conflict trace
`s.dm(type=None, contains=None)`	DM by activation
`s.chunk(name)`	slots, activation, Sji
`s.productions()`	productions, utilities
`s.associations(source, type, learned)`	Sji skill → operators
`s.tasks()`	task list
`s.results / s.results_table()`	list / DataFrame
`s.source(task) / s.to_python(task)`	source / as Python

Plotting

`s.plot_results(average_window, labels)`	one line per run
`s.plot_operator_activations()`	last trial (see below)
`s.plot_graph(level=2)`	PRIMs graph
`read_batch(path) / plot_batch(src)`	batch output ±1 SE
`figure(...) / show_figure(fig)`	plots take ax=

Cell magics

`%%prims [session] [--reset] [--python]`	cell in PRIMs syntax
`%%pyprims [session] [--reset]`	cell defining task = Task(...)

Settings: `s.track_operator_activations = True` before the trial, for the activation plot; `s.warn_failed_trials = False` silences NoOperatorWarning; `Session(trace_level=3, echo_print=False)`.